SECURE CODING REVIEW REPORT

CodeAlpha Cyber Security Internship | Task 3

Application: Python SQLite Login Authentication System
Review methods: Bandit static analysis, manual inspection, and user-reported functional tests
Report date: 27 October 2026

1. Executive Summary

Two Python authentication implementations were compared: vulnerabilities_app.py (intentionally insecure) and main.py (hardened). Bandit scanned 83 lines in the vulnerable application and reported two findings; it scanned 223 lines in the secure application and reported zero findings. The user also confirmed that a short password was accepted in the vulnerable application but rejected in the secure application, and that the secure application locked the account after five failed login attempts. These are test observations, not proof that the hardened application is free of all vulnerabilities.

2. Scope and Methodology

Scope: authentication, password policy, SQL query construction, and brute-force controls. Static analysis was performed using Bandit. Manual findings are based on the provided secure source code, Bandit excerpts from the vulnerable code, and user-reported test behavior. A complete source-level audit of the scanned vulnerabilities_app.py file was not available; additional findings should not be represented as verified.

3. Bandit Scan Comparison

Metric	Vulnerable	Secure
Scanned file	vulnerabilities_app.py	main.py
Lines scanned	83	223
Low severity	1	0
Medium severity	1	0
High severity	0	0
Files skipped	0	0

4. Confirmed Static Analysis Findings

Finding A - B105: Possible hardcoded password
Location: vulnerabilities_app.py, line 12. Severity: Low; confidence: Medium; CWE-259. The scan identified an ADMIN_PASSWORD constant assigned a literal password. **Risk:** credentials can be disclosed through access to source code. **Remediation:** remove hardcoded admin passwords; store salted password hashes and provision administrator accounts securely.

Finding B - B608: SQL query constructed with string concatenation
Location: vulnerabilities_app.py, line 87. Severity: Medium; confidence: Low; CWE-89. The scan identified a SELECT query built by concatenating user input. **Risk:** potential SQL injection. **Remediation:** use SQLite parameter placeholders and bind user input as parameters; verify passwords using stored hashes separately. The scanner identifies a possible injection vector, not a confirmed successful exploit.

5. Manual Review and Functional Testing

Password policy: User testing confirmed that the vulnerable app accepted password "123", while the secure app rejected it. The reviewed secure source enforces a minimum of 12 characters and maximum of 128 characters. **Recommendation:** also screen common/compromised passwords.

Login throttling: User testing confirmed account lockout after five incorrect passwords in the secure app. The secure source sets MAX_ATTEMPTS=5 and LOCK_TIME=60 seconds. **Recommendation:** add monitoring and consider controls against lockout abuse.

Secure-source controls: The provided main.py uses PBKDF2-HMAC-SHA256 with a random salt, constant-time digest comparison, and parameterized SQLite queries. These are positive design features; review and testing remain necessary for production use.

6. Before-and-After Summary

The vulnerable implementation contains a hardcoded password and potentially unsafe SQL construction, while the reviewed secure implementation uses hashed passwords and parameterized SQL. User-reported testing additionally demonstrates improved password policy and login-attempt protection. Bandit found 2 issues in the vulnerable file and 0 in the secure file; these counts describe tool findings only.

7. Recommended Remediation Steps

1. Remove source-embedded passwords and securely provision accounts.
2. Replace concatenated SQL with parameterized queries.
3. Store only salted, slow password hashes.
4. Enforce minimum password length and reject common passwords.
5. Implement and test rate limiting and account lockout safely.
6. Restrict database file permissions and avoid exposing sensitive information.
7. Re-run Bandit and functional security tests after every relevant change.

8. Limitations

The report does not claim a demonstrated SQL injection exploit. Static analysis cannot detect every application logic flaw. The test outcomes for weak passwords and account lockout were reported by the user rather than independently captured in this report. Screenshots of those tests may be attached as supplementary evidence.

9. Conclusion

The exercise demonstrates how automated static analysis and manual testing complement each other. Two Bandit findings were recorded in the vulnerable app, and no Bandit findings were recorded in the hardened app. The hardened code includes meaningful security protections, but further testing is advisable before any real-world deployment.

Evidence and References

Evidence: vulnerable_scan(1).txt (Bandit B105/B608, 83 scanned lines); secure_scan(1).txt (zero issues, 223 scanned lines); reviewed main.py source; user-reported login and password tests.
References: Bandit security scanner documentation (bandit.readthedocs.io); CWE-259 and CWE-89 (cwe.mitre.org); OWASP Authentication Cheat Sheet (cheatsheetseries.owasp.org).